

Name: Sahil Santosh More

PRN:202501040032

PRACTICAL: 04

4.1.1. Pandas - series creation and manipulation

03:12

Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the groupby and mean() operations.

Hint: You can group the Series based on whether each number is even or odd.

Input Format:

- The user should enter a list of numbers separated by space when prompted.

Output Format:

- The program should display the mean of even and odd numbers separately.
- Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.

Refer to the sample test cases for better understanding regarding input and output format.

```
# import pandas as pd
```

```
# # Take inputs from the user to create a list of numbers
```

```
# numbers = list(map(int, input().split()))
```

```
# grouped =
```

```
# # Display the mean of even and odd numbers with labels
```

```
# grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
```

```
# print("Mean of even and odd numbers:")
```

```
# print(grouped)
```

```
import pandas as pd
```

```
# Take inputs from the user to create a list of numbers
```

```
numbers = list(map(int, input().split()))
```

```
# Create a Pandas series from the list of numbers
```

```
s = pd.Series(numbers)
```

```
# Grouping by even and odd numbers and calculating the mean
```

```
grouped = s.groupby(s % 2 == 0).mean()
```

```
# Display the mean of even and odd numbers with labels
```

```
grouped.index = ['Even' if is_even else 'Odd' for is_even in grouped.index]
```

```
print("Mean of even and odd numbers:")
```

```
print(grouped)
```

The screenshot displays a Jupyter Notebook interface with a light blue sidebar on the left labeled 'Explorer' and a light blue sidebar on the right labeled 'Debugger'. The main content area shows the results of a pandas operation. At the top, there are two summary boxes: 'Average time' with '0.007 s' and '7.00 ms', and 'Maximum time' with '0.017 s' and '17.00 ms'. To the right of these boxes, there are two green checkmarks indicating '3 out of 3 shown test case(s) passed' and '3 out of 3 hidden test case(s) passed'. Below these, there is a table with two columns: 'Expected output' and 'Actual output'. The table contains three rows of test cases. The first row, 'Test case 1', is expanded and shows the following data: 'Expected output' and 'Actual output' both show a list of numbers '1 2 3 4 5 6 7 8 9 10'. Below this, the 'Mean of even and odd numbers:' is shown as 'Mean of even and odd numbers:'. The 'Odd' row shows 'Odd: 5.0' and the 'Even' row shows 'Even: 6.0'. The 'dtype' is shown as 'dtype: float64'. The second row, 'Test case 2', and the third row, 'Test case 3', are collapsed and show '5 ms'.

Test case	Expected output	Actual output
Test case 1	1 2 3 4 5 6 7 8 9 10 Mean of even and odd numbers: Odd: 5.0 Even: 6.0 dtype: float64	1 2 3 4 5 6 7 8 9 10 Mean of even and odd numbers: Odd: 5.0 Even: 6.0 dtype: float64
Test case 2		
Test case 3		

4.1.2. Dictionary to dataframe

07:26

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.
- Display the DataFrame after modifying the column.

Delete a column:

- Remove the **Age** column.
- Display the DataFrame after deleting the column.

4.1.3. Student Information

01:49

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:

Refer to the displayed test cases for better understanding.

```
import pandas as pd
```

```
# Read the text file into a DataFrame
```

```
file = input()
```

```
data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age", "Grade"])
```

```
# Display the first five rows
```

```
print("First five rows:")
```

```
print(data.head())
```

```
# Calculate the average age (2 decimal places)
```

```
avg_age = round(data["Age"].mean(), 2)
```

```
print("Average age:", avg_age)
```

```
# Filter students with grade up to 'B' (A and B)
```

```
filtered = data[data["Grade"].isin(['A', 'B'])]
```

```
print("Students with a grade up to B")
```

```
print(filtered)
```

The screenshot displays a coding interface with a terminal and test cases. The terminal shows the execution of a script that prints the first five rows of a dataset, calculates the average age, and filters students with a grade of 'A' or 'B'. The test results indicate that all test cases passed.

Test Results Summary:

- Average time: 0.028 s (27.50 ms)
- Maximum time: 0.040 s (40.00 ms)
- 1 out of 1 shown test case(s) passed
- 1 out of 1 hidden test case(s) passed

Test Case 1 (40 ms):

Expected output	Actual output
studentdata.txt	studentdata.txt
First five rows:	First five rows:
...	...
0 John 25 A	0 John 25 A
1 Alin 22 B	1 Alin 22 B
2 Emma 24 A	2 Emma 24 A
3 Mike 23 C	3 Mike 23 C
4 Sony 21 B	4 Sony 21 B
Average age: 23.29	Average age: 23.29
Students with a grade up to B:	Students with a grade up to B:
...	...
0 John 25 A	0 John 25 A
1 Alin 22 B	1 Alin 22 B
2 Emma 24 A	2 Emma 24 A
4 Sony 21 B	4 Sony 21 B
5 Amia 26 A	5 Amia 26 A

Navigation: < Prev, Reset, Submit, Next >

4.2.1. Month with the Highest Total Sales

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
# Find the month with the highest total sales
```

```
df['Date'] = pd.to_datetime(df['Date'])
```

```
df['Month'] = df['Date'].dt.to_period('M').astype(str) # Convert to YYYY-MM format
```

```
df['Total Sales'] = df['Quantity'] * df['Price']
```

```
monthly_sales = df.groupby('Month')['Total Sales'].sum()
```

```
# Find the month with the highest total sales
```

```
best_month = monthly_sales.idxmax()
```

```
highest_sales = monthly_sales.max()
```

```
print(f"Best month: {best_month}")
```

```
print(f"Total sales: ${highest_sales:.2f}")
```


Average time

0.082 s

82.00 ms

Maximum time

0.182 s

182.00 ms

✓ 1 out of 1 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1

182 ms

Debug

Expected output

Actual output

sales_data.csv	sales_data.csv
Best month: 2025-01	Best month: 2025-01
Total sales: \$1210.00	Total sales: \$1210.00

Terminal

Test cases

< Prev

Reset

Submit

Next >

4.2.2. Best Selling Product

02:08

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
# Find the product with the highest total quantity sold
```

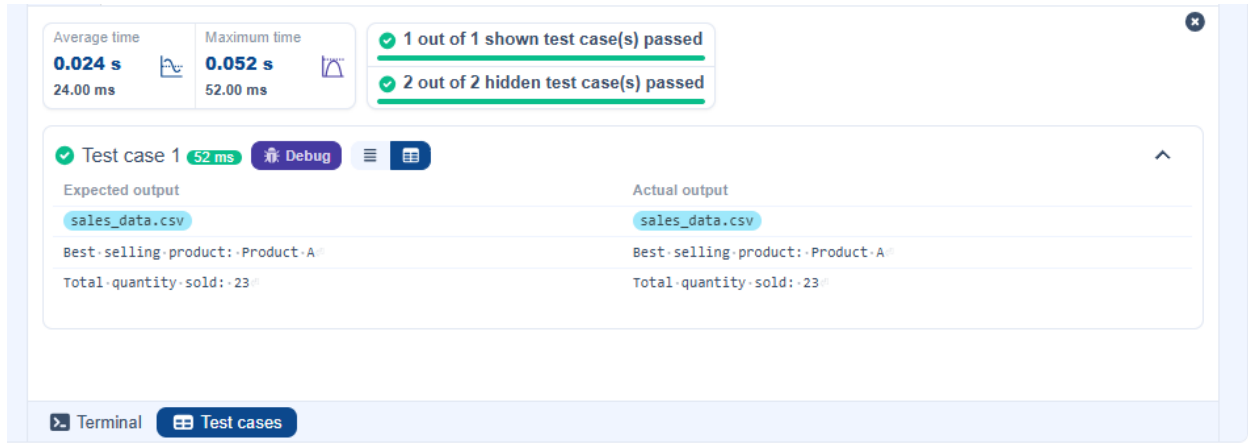
```
best_product = df.groupby('Product')['Quantity'].sum().idxmax()
```

```
highest_quantity = df.groupby('Product')['Quantity'].sum().max()
```

```
# Display the result
```

```
print(f"Best selling product: {best_product}")
```

```
print(f"Total quantity sold: {highest_quantity}")
```



4.2.3. City that Sold the Most Products

01:45

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,2,15,Chicago

2025-01-03,Product A,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product B,5,15,Los Angeles

2025-01-05,Product A,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

```
import pandas as pd
```

```
# Prompt the user for the file name
```

```
file_name = input()
```

```
# Load the data
```

```
df = pd.read_csv(file_name)
```

```
# Group by City and calculate total quantity
```

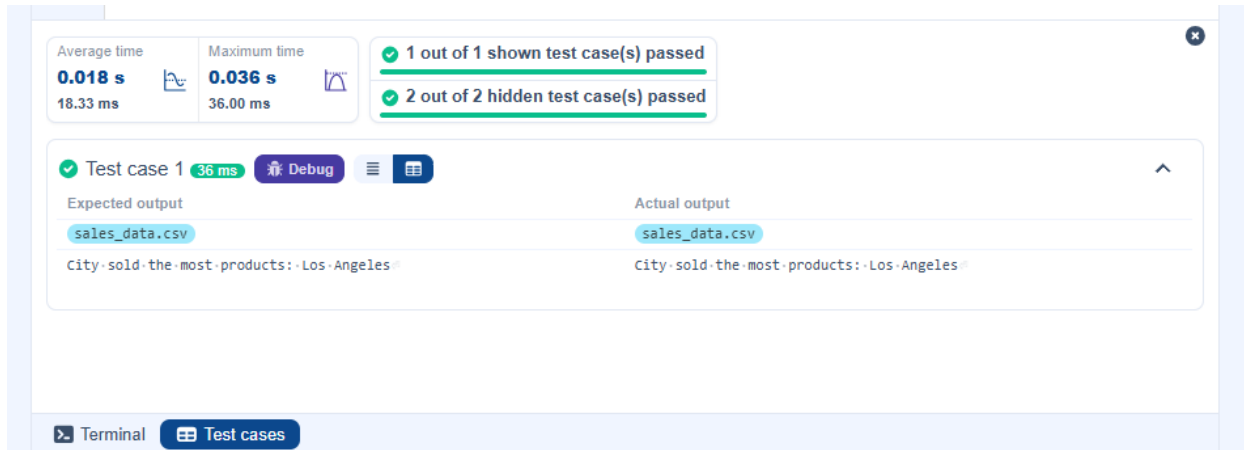
```
city_sales = df.groupby('City')['Quantity'].sum()
```

```
# Find the city with the highest total quantity sold
```

```
best_city = city_sales.idxmax()
```

```
# Display the result
```

```
print(f"City sold the most products: {best_city}")
```



4.2.4. Most Frequently Sold Product Pairs

01:25

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,2,15,Chicago

2025-01-03,Product A,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product B,5,15,Los Angeles

2025-01-05,Product A,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Explanation:

Transactions:

- **2025-01-01:** Product A, Product B
- **2025-01-02:** Product A, Product C
- **2025-01-03:** Product B, Product A
- **2025-01-04:** Product C, Product B
- **2025-01-05:** Product A, Product C

Now, let's count how often the pairs of products appear together:

- **Product A and Product B:** Appear in transactions on 2025-01-01 and 2025-01-03.
- **Product A and Product C:** Appear in transactions on 2025-01-02 and 2025-01-05.
- **Product B and Product C:** Appears in transactions on 2025-01-04.

Most Frequent Product Combinations:

- **Product A and Product B** (2 times)
- **Product A and Product C** (2 times)

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

```
import pandas as pd
```

```
from itertools import combinations
from collections import Counter

# Prompt user to input the file name
file_name = input()

# Read data from the specified CSV file
df = pd.read_csv(file_name)

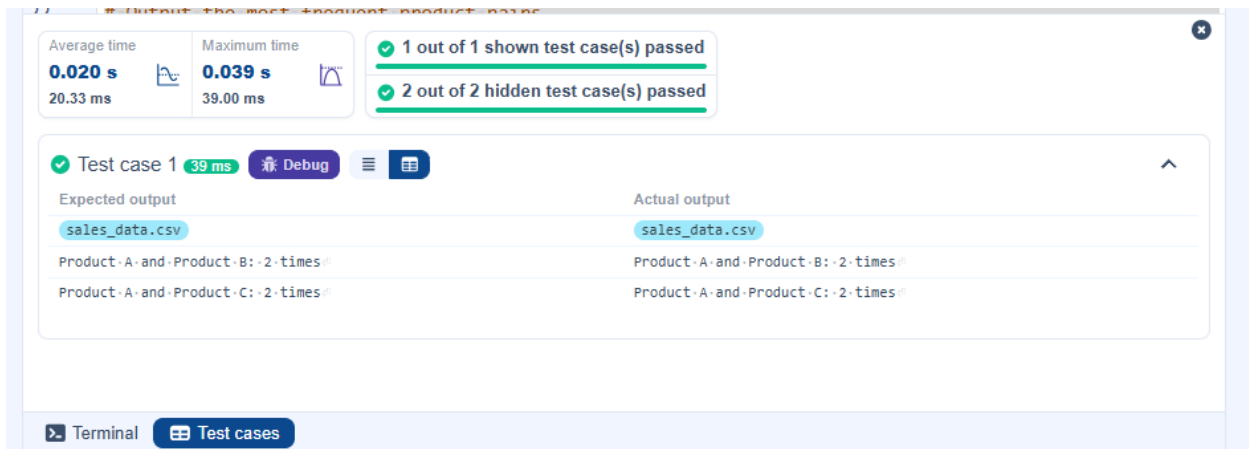
# write the code
pair_counter = Counter()

# Group by Date and generate product pairs
for _, group in df.groupby('Date'):
    products = group['Product'].tolist()
    pairs = combinations(sorted(products), 2)
    pair_counter.update(pairs)

# Find maximum frequency
max_count = max(pair_counter.values())

# Output the most frequent product pairs
for pair, count in pair_counter.items():
    if count == max_count:
```

```
print(f"{pair[0]} and {pair[1]}: {count} times")
```



4.2.5. Titanic Dataset Analysis and Data Cleaning

01:36

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

1. Display the first 5 rows of the dataset.
2. Display the last 5 rows of the dataset.
3. Get the shape of the dataset (number of rows and columns).
4. Get a summary of the dataset (using `.info()`).
5. Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
6. Check for missing values and display the count of missing values for each column.
7. Fill missing values in the 'Age' column with the median age.
8. Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
9. Drop the 'Cabin' column due to many missing values.
10. Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC
17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

1. Display the first 5 rows of the dataset

```
print(data.head())
```

2. Display the last 5 rows of the dataset

```
print(data.tail())
```

3. Get the shape of the dataset

```
print(data.shape)
```

4. Get a summary of the dataset (info)

```
print(data.info())
```

5. Get basic statistics of the dataset

```
print(data.describe())
```

6. Check for missing values

```
print(data.isnull().sum())
```

7. Fill missing values in the 'Age' column with the median age

```
data['Age'] = data['Age'].fillna(data['Age'].median())
```

8. Fill missing values in the 'Embarked' column with the mode

```
data['Embarked'] = data['Embarked'].fillna(data['Embarked'].mode()[0])
```

9. Drop the 'Cabin' column due to many missing values

```
data = data.drop(columns=['Cabin'])
```

10. Create a new column 'FamilySize' by adding 'SibSp' and 'Parch'

`data['FamilySize'] = data['SibSp'] + data['Parch']`

The screenshot displays a Jupyter Notebook interface with a light blue theme. At the top, a status bar indicates 'Average time 0.166 s' and 'Maximum time 0.166 s'. A green checkmark icon is followed by the text '1 out of 1 shown test case(s) passed'. Below this, a table compares two dataframes side-by-side. The table has two main columns, each containing a list of statistics for the dataframes. The statistics include count, mean, std, min, 25%, 50%, 75%, max, and dtype. The dataframes are identical, with the same values for each statistic. Below the table, a red bar highlights the difference in the number of rows and columns between the two dataframes. The bottom of the interface shows a terminal and test cases section, with buttons for 'Terminal', 'Test cases', 'Prev', 'Reset', 'Submit', and 'Next'. The system clock in the bottom right corner shows 9:37 PM on 4/24/2026.

Statistic	Left DataFrame	Right DataFrame
count	891.000000 891.000000	891.000000 891.000000
mean	446.000000 0.383838 2.308642 0.523008 0.38 1594 32.204208	446.000000 0.383838 2.308642 0.523008 0.38 1594 32.204208
std	257.353842 0.486592 0.836071 1.102743 0.80 6057 49.693429	257.353842 0.486592 0.836071 1.102743 0.80 6057 49.693429
min	1.000000 0.000000 1.000000 0.000000 0.00 0000 0.000000	1.000000 0.000000 1.000000 0.000000 0.00 0000 0.000000
25%	223.500000 0.000000 2.000000 0.000000 0.00 0000 7.910400	223.500000 0.000000 2.000000 0.000000 0.00 0000 7.910400
50%	446.000000 0.000000 3.000000 0.000000 0.00 0000 14.454200	446.000000 0.000000 3.000000 0.000000 0.00 0000 14.454200
75%	668.500000 1.000000 3.000000 1.000000 0.00 0000 31.000000	668.500000 1.000000 3.000000 1.000000 0.00 0000 31.000000
max	891.000000 1.000000 3.000000 8.000000 6.00 0000 512.329200	891.000000 1.000000 3.000000 8.000000 6.00 0000 512.329200
[8 rows x 7 columns]	[8 rows x 7 columns]	[8 rows x 7 columns]
PassengerId	0	0
Survived	0	0
Pclass	0	0
Name	0	0
Sex	0	0
Age	177	177
SibSp	0	0
Parch	0	0
Ticket	0	0
Fare	0	0
Cabin	687	687
Embarked	2	2
dtype	int64	int64

4.2.6. Titanic Dataset Analysis and Data Cleaning - 2

02:18

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC
17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S
4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S
5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S
6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q
7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S
8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S
9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S
10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.

```
import pandas as pd
```

```
import numpy as np
```

```
# Load the Titanic dataset
```

```
data = pd.read_csv('Titanic-Dataset.csv')
```

```
data['FamilySize'] = data['SibSp'] + data['Parch']
```

```
# 1. Create a new column 'IsAlone' (1 if alone, 0 otherwise)
```

```
data['IsAlone'] = np.where(data['FamilySize'] == 0, 1, 0)
```

```
# 2. Convert 'Sex' to numeric (male: 0, female: 1)
```

```
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
```

```
# 3. One-hot encode the 'Embarked' column
```

```
data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
```

4. Get the mean age of passengers

```
print(data['Age'].mean())
```

5. Get the median fare of passengers

```
print(data['Fare'].median())
```

6. Get the number of passengers by class

```
print(data['Pclass'].value_counts())
```

7. Get the number of passengers by gender

```
print(data['Sex'].value_counts())
```

8. Get the number of passengers by survival status

```
print(data['Survived'].value_counts())
```

9. Calculate the survival rate

```
print(data['Survived'].mean())
```

10. Calculate the survival rate by gender

```
print(data.groupby('Sex')['Survived'].mean())
```

The screenshot shows a Jupyter Notebook interface with a successful test case. At the top, a status bar indicates '1 out of 1 shown test case(s) passed' with a green checkmark. Below this, the test case details are shown: 'Test case 1' with a duration of '79 ms' and a 'Debug' button. The main content area displays a comparison between 'Expected output' and 'Actual output' for a pandas groupby operation. The output is a Series of mean survival rates for each sex, with the following values: 29.69911764705882, 14.4542, 3.491, 1.216, 2.184, 0.577, 1.314, 0.549, 1.342, 0.3838383838383838, 0.188908, and 0.742038. The dtype for the first nine values is 'int64', and for the last three, it is 'float64'.

Expected output	Actual output
29.69911764705882	29.69911764705882
14.4542	14.4542
3.491	3.491
1.216	1.216
2.184	2.184
Name: Pclass, dtype: int64	Name: Pclass, dtype: int64
0.577	0.577
1.314	1.314
Name: Sex, dtype: int64	Name: Sex, dtype: int64
0.549	0.549
1.342	1.342
Name: Survived, dtype: int64	Name: Survived, dtype: int64
0.3838383838383838	0.3838383838383838
Sex	Sex
0.188908	0.188908
1.742038	1.742038
Name: Survived, dtype: float64	Name: Survived, dtype: float64

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

03:10

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).

7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

PassengerId,Survived,Pclass,Name,Sex,Age,SibSp,Parch,Ticket,Fare,Cabin,Embarked

1,0,3,"Braund, Mr. Owen Harris",male,22,1,0,A/5 21171,7.25,,S

2,1,1,"Cumings, Mrs. John Bradley (Florence Briggs Thayer)",female,38,1,0,PC
17599,71.2833,C85,C

3,1,3,"Heikkinen, Miss. Laina",female,26,0,0,STON/O2. 3101282,7.925,,S

4,1,1,"Futrelle, Mrs. Jacques Heath (Lily May Peel)",female,35,1,0,113803,53.1,C123,S

5,0,3,"Allen, Mr. William Henry",male,35,0,0,373450,8.05,,S

6,0,3,"Moran, Mr. James",male,,0,0,330877,8.4583,,Q

7,0,1,"McCarthy, Mr. Timothy J",male,54,0,0,17463,51.8625,E46,S

8,0,3,"Palsson, Master. Gosta Leonard",male,2,3,1,349909,21.075,,S

9,1,3,"Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)",female,27,0,2,347742,11.1333,,S

10,1,2,"Nasser, Mrs. Nicholas (Adele Achem)",female,14,1,0,237736,30.0708,,C

Note: Refer to the visible test case for better reference.


```
import pandas as pd

import numpy as np

# Load the Titanic dataset

data = pd.read_csv('Titanic-Dataset.csv')

data['FamilySize'] = data['SibSp'] + data['Parch']

data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)

data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)

# 1. Calculate the survival rate by class

print(data.groupby('Pclass')['Survived'].mean())


# 2. Calculate the survival rate by embarked location

print(data.groupby('Embarked_S')['Survived'].mean())


# 3. Calculate the survival rate by family size

print(data.groupby('FamilySize')['Survived'].mean())


# 4. Calculate the survival rate by being alone

print(data.groupby('IsAlone')['Survived'].mean())


# 5. Get the average fare by class

print(data.groupby('Pclass')['Fare'].mean())


# 6. Get the average age by class

print(data.groupby('Pclass')['Age'].mean())
```

7. Get the average age by survival status

```
print(data.groupby('Survived')['Age'].mean())
```

8. Get the average fare by survival status

```
print(data.groupby('Survived')['Fare'].mean())
```

9. Get the number of survivors by class

```
print(data[data['Survived'] == 1]['Pclass'].value_counts())
```

10. Get the number of non-survivors by class

```
print(data[data['Survived'] == 0]['Pclass'].value_counts())
```

Average time: 0.103 s, 103.00 ms. Maximum time: 0.103 s, 103.00 ms. 1 out of 1 shown test case(s) passed.

Left Output:

```
5 ---- 0.136364
6 ---- 0.333333
7 ---- 0.000000
10 ---- 0.000000
Name: Survived, dtype: float64
IsAlone:
0 ---- 0.505650
1 ---- 0.303538
Name: Survived, dtype: float64
Pclass:
1 ---- 84.154687
2 ---- 20.662183
3 ---- 13.675550
Name: Fare, dtype: float64
Pclass:
1 ---- 38.233441
2 ---- 29.877630
3 ---- 25.140620
Name: Age, dtype: float64
Survived:
0 ---- 30.626179
1 ---- 28.343690
Name: Age, dtype: float64
Survived:
0 ---- 22.117887
1 ---- 48.395408
Name: Fare, dtype: float64
1 ---- 136
3 ---- 119
2 ---- 87
Name: Pclass, dtype: int64
3 ---- 372
2 ---- 97
1 ---- 80
```

Right Output:

```
5 ---- 0.136364
6 ---- 0.333333
7 ---- 0.000000
10 ---- 0.000000
Name: Survived, dtype: float64
IsAlone:
0 ---- 0.505650
1 ---- 0.303538
Name: Survived, dtype: float64
Pclass:
1 ---- 84.154687
2 ---- 20.662183
3 ---- 13.675550
Name: Fare, dtype: float64
Pclass:
1 ---- 38.233441
2 ---- 29.877630
3 ---- 25.140620
Name: Age, dtype: float64
Survived:
0 ---- 30.626179
1 ---- 28.343690
Name: Age, dtype: float64
Survived:
0 ---- 22.117887
1 ---- 48.395408
Name: Fare, dtype: float64
1 ---- 136
3 ---- 119
2 ---- 87
Name: Pclass, dtype: int64
3 ---- 372
2 ---- 97
1 ---- 80
```

Navigation: < Prev, Reset, Submit, Next >